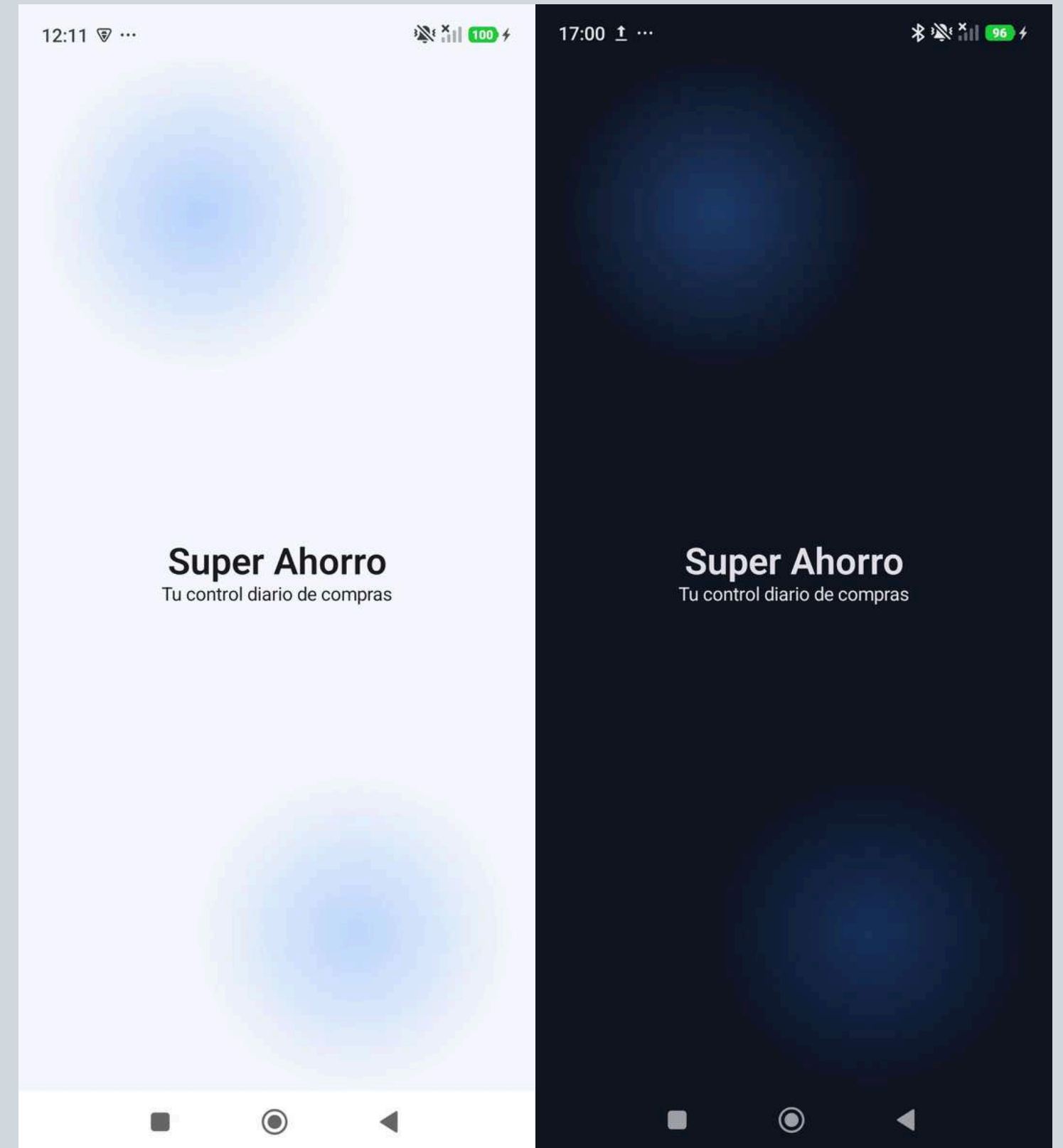


Super Ahorro



Caparroz, Sebastian
Ruiz Notari, Leonel

MainActivity.kt

```
package com.undef.superahorro.caparrozruiz

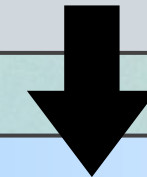
import android.os.Bundle
import androidx.activity.ComponentActivity //Para trabajar con Jetpack Compose
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import com.undef.superahorro.caparrozruiz.ui.SuperAhorroApp

1 Usage
class MainActivity : ComponentActivity() {
    //Punto de entrada de la Activity donde se define la UI mediante setContent
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()//Habilita el diseño de pantalla completa
        setContent {
            // Se utiliza el patrón Single Activity: SuperAhorroApp gestiona el NavHost
            SuperAhorroApp()
        }
    }
}
```

MainActivity



SuperAhorroApp



SuperAhorroTheme



SuperAhorroNavHost



Pantallas

SuperAhorroNavHost.kt

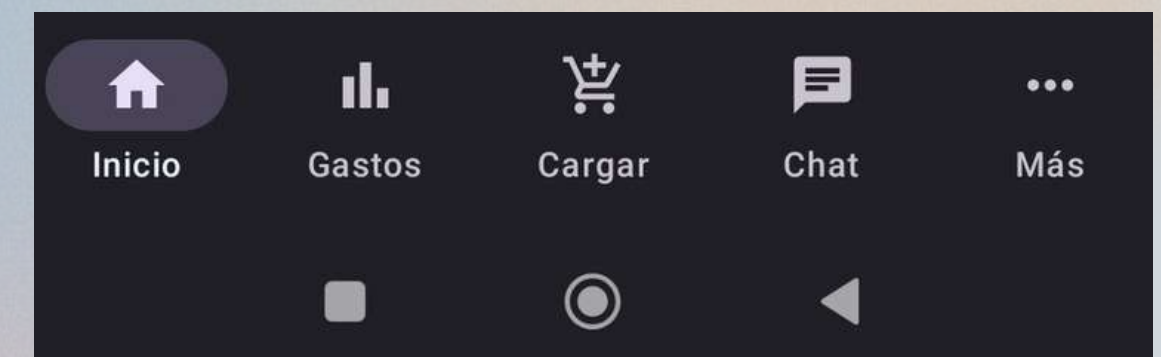
Declaración de variables

```
@Composable
fun SuperAhorroNavHost() {
    val navController = rememberNavController() //Gestiona el historial y los cambios de pantallas
    val currentRoute = navController.currentBackStackEntryAsState().value?.destination?.route
    val showBottomBar = currentRoute in mainRoutes //Determina si mostrar la BottomBar o no
```

- Uso de Lista mainRoutes para mayor escalabilidad
- Uso de saveState para que el usuario no pierda su progreso al cambiar de pantallas

Lógica de BottomBar

```
Scaffold(
    bottomBar = {
        if (showBottomBar) {
            NavigationBar {
                bottomNavItems.forEach { item ->
                    NavigationBarItem(
                        selected = currentRoute == item.route,
                        onClick = {
                            //Navega sin acumular pantallas en el back stack
                            navController.navigate(item.route) {
                                popUpTo(route = AppRoute.Home) {
                                    inclusive = false
                                }
                                launchSingleTop = true
                                restoreState = false
                            }
                        },
                        icon = { Icon(imageVector = item.icon, contentDescription = null) },
                        label = { Text(text = stringResource(id = item.labelRes)) }
                    )
                }
            }
        }
    }
)
```



SuperAhorroNavHost.kt

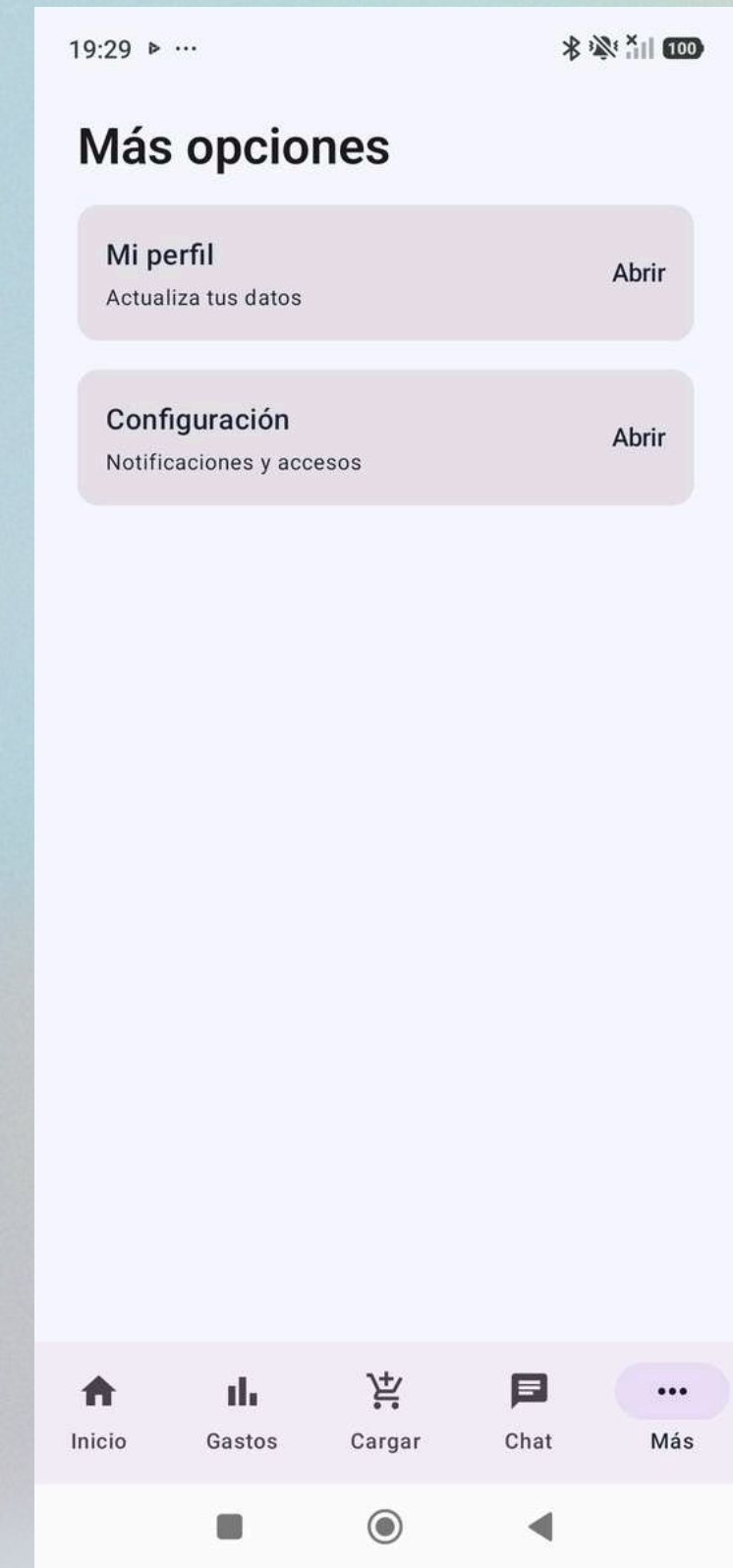
Definición de navegación

```
//Contenedor que vincula las rutas con sus respectivos Composables  
NavHost(  
    navController = navController,  
    startDestination = AppRoute.Splash,  
    modifier = Modifier.padding( paddingValues = innerPadding)
```

Destinos

```
composable( route = AppRoute.Settings) {  
    SettingsScreen(  
        onLogoutClick = {  
            //Reinicia el flujo de navegacion hacia el Login tras cerrar sesion  
            navController.navigate( route = AppRoute.Login) {  
                popUpTo( route = AppRoute.Home) { inclusive = true }  
                launchSingleTop = true  
            }  
        }  
    )  
}  
  
composable( route = AppRoute.More) {  
    MoreScreen(  
        onOpenProfile = { navController.navigate( route = AppRoute.Profile) },  
        onOpenSettings = { navController.navigate( route = AppRoute.Settings) }  
    )  
}
```

- Las pantallas no conocen al NavController. Solo notifican eventos, delegando la responsabilidad de la navegación al NavHost.



Intents (NewPurchaseScreen.kt)

```
Row(horizontalArrangement = Arrangement.spacedBy( space = 8.dp)) {  
    OutlinedButton(  
        onClick = {  
            //Crea un intent para abrir la galeria  
            val intent = Intent( action = Intent.ACTION_PICK).apply { type = "image/*" }  
            //Lanza la actividad externa utilizando el contexto de la aplicación  
            context.startActivity(intent)  
            viewModel.setTicketStatus("Galeria")  
        }  
    ) {  
        Text(text = stringResource("Galeria"))  
    }  
    OutlinedButton(  
        onClick = {  
            //Crea un intent para abrir la camara  
            val intent = Intent( action = "android.media.action.IMAGE_CAPTURE")  
            //Delega la tarea a la actividad externa  
            context.startActivity(intent)  
            viewModel.setTicketStatus("Cámara")  
        }  
    ) {  
        Text(text = stringResource("Cámara"))  
    }  
}  
  
if (uiState.ticketStatus.isNotBlank()) {  
    Text(  
        text = stringResource(  
            "Adjunto simulado: %1$s",  
            ...formatArgs = uiState.ticketStatus  
        ),  
        style = MaterialTheme.typography.bodySmall  
    )  
}
```

- Intents Implícitos



Composable (HomeScreen.kt)

Definición del Composable

```
@Composable
fun HomeScreen(
    onOpenHistory: () -> Unit,
    viewModel: HomeViewModel = viewModel() //ViewModel que trae datos
) {
    //Observa el estado del viewModel y actualiza la UI
    val uiState by viewModel.uiState.collectAsStateWithLifecycle()
    //Para que use la configuración del celular (idioma, tema)
    val locale = LocalConfiguration.current.locales[0]
```

- La pantalla no decide a donde ir, solo dice que presionaron un boton
- Estado reactivo con uiState

Uso de LazyColumn

```
LazyColumn( //Para que solo renderice los elementos en pantalla
    modifier = Modifier
        .fillMaxSize()
        .padding(all = 16.dp),
    verticalArrangement = Arrangement.spacedBy(space = 12.dp)
) {
    item {
        Text( //Texto con uso de strings.xml
            text = stringResource(id = R.string.home_welcome, ...formatArgs = uiState.userName),
            style = MaterialTheme.typography.headlineMedium
        )
    }
    item {
```

- Uso de LazyColumn
- Uso de stringResource

Composable (HomeScreen.kt)

```
//Itera sobre la lista de datos para generar componentes de forma dinamica
items( items = uiState.purchases, key = { it.id }) { purchase ->
    PurchaseItem(
        purchase = purchase,
        currencyLabel = stringResource( id = R.string.common_currency_symbol)
    )
}

item {
    //Accion de navegacion delegada a NavHost
    Button(onClick = onOpenHistory, modifier = Modifier.fillMaxWidth()) {
        Text(text = stringResource( id = R.string.home_open_history_button))
    }
}
```

- Uso de key para asignar identidad unica a cada compra
- La lista ya viene prefiltrada desde el ViewModel
- El boton no conoce la ruta



ViewModel (HomeViewModel.kt)

- Uso de HomeUiState para garantizar que la UI sea reactiva
- Uso de _uiState (Mutable) donde solo el ViewModel puede cambiar los datos
- Uso de uiState (Read-Only) evitando que la vista altere el estado directamente
- Con viewModelScope nos aseguramos que las tareas de actualización solo se ejecutan en el ámbito vinculado al ciclo de vida del ViewModel
- Uso de collectLatest para que la aplicación reaccione a los cambios

```
//Agrupar los datos en una data class ayuda a la UI a reaccionar a un solo objeto
data class HomeUiState(
    val userName: String = "",
    val monthlyBudget: Double = 0.0,
    val purchases: List<Purchase> = emptyList()
)

class HomeViewModel : ViewModel() {
    //Referencia al repository, ViewModel no sabe como se guardan los datos, solo los pide
    private val repository = FakeWalletRepository
    private val currentUser = repository.getCurrentUser()

    //Privado para que no se pueda modificar desde fuera
    private val _uiState = MutableStateFlow(
        value = HomeUiState( //Inicializado con datos basicos del usuario
            userName = currentUser.name,
            monthlyBudget = currentUser.monthlyBudget,
            purchases = emptyList()
        )
    )

    //Version de solo lectura para la UI
    val uiState: StateFlow<HomeUiState> = _uiState.asStateFlow()
}
```

```
init {
    viewModelScope.launch {
        //Si los datos del repository cambian, se actualiza el estado
        repository.purchasesFlow().collectLatest { purchases ->

            _uiState.value = _uiState.value.copy(purchases = purchases.take( n = 5))
        }
    }
}
```

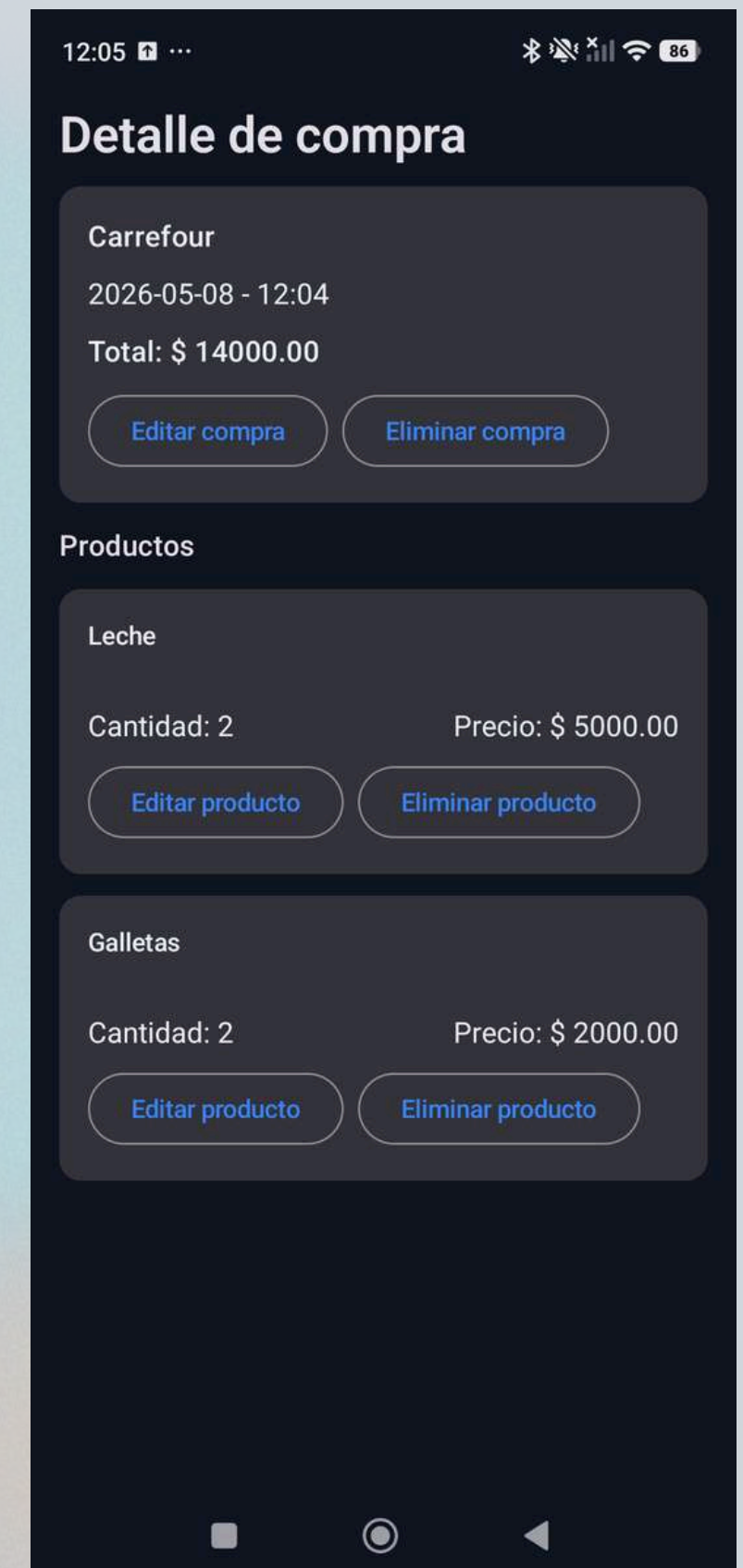
Repository (FakeWalletRepository.kt)

```
private val user = User(  
    name = "Martin Perez",  
    email = "martin.perez@email.com",  
    city = "Cordoba",  
    monthlyBudget = 265000.0  
)
```

```
private val initialPurchases = listOf(  
    Purchase(id = "P001", market = "Carrefour", date = "2026-05-01", time = "10:15", total = 38650.0),  
    Purchase(id = "P002", market = "Disco", date = "2026-04-29", time = "18:40", total = 23400.0),  
    Purchase(id = "P003", market = "Coto", date = "2026-04-27", time = "12:10", total = 41200.0),  
    Purchase(id = "P004", market = "Dia", date = "2026-04-24", time = "20:05", total = 15780.0)  
)
```

- Simulación de datos reales

```
1 Usage  
fun addPurchase(purchase: Purchase, products: List<Product>) {  
    //Guarda la compra al inicio de la lista y vincula sus productos  
    purchasesState.value = listOf(purchase) + purchasesState.value  
    if (products.isNotEmpty()) {  
        productsState.value = productsState.value + (purchase.id to products)  
    }  
    clearDraftProducts()//Limpiar el borrador  
}
```



Flujo de Usuario

Inicio de Sesión

12:17

85

Hola, Martin Perez

Presupuesto mensual estimado

\$ 265000.00

Últimas compras

Carrefour

2026-05-01

\$ 38650.00

Disco

2026-04-29

\$ 23400.00

Coto

2026-04-27

\$ 41200.00

Dia

2026-04-24

\$ 15780.00

Ver historial

Inicio

Gastos

Cargar

Chat

Más

Detalle de Compra

12:25

84

Detalle de compra

Carrefour

2026-05-08 - 12:24

Total: \$ 15000.00

Editar compra

Eliminar compra

Productos

Leche

Cantidad: 2

Precio: \$ 5000.00

Editar producto

Eliminar producto

Galletas

Cantidad: 2

Precio: \$ 2500.00

Editar producto

Eliminar producto

Cargar Compra

12:32

83

Presupuesto mensual estimado

\$ 265000.00

Últimas compras

Test

2026-05-08

\$ 10000.00

Carrefour

2026-05-08

\$ 15000.00

Carrefour

2026-05-01

\$ 38650.00

Disco

2026-04-29

\$ 23400.00

Coto

2026-04-27

\$ 41200.00

Ver historial

Inicio

Gastos

Cargar

Chat

Más

Cerrar Sesión

12:15

85

Iniciar sesión

Correo electrónico

Contraseña

Ingresar

Olvidé mi contraseña

No tengo cuenta

Muchas
Gracias